

AFRILANG Stdlib

std/c/mlcrossval

Français. Bibliothèque AFRILANG 'std/c/mlcrossval' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : crosSum, crosMean, crosMin, crosMax, crosVar, crosStd, crosNorm.

```
'import "std/c/mlcrossval"' · 'use mlcrossval'
```

```
- 'crosSum(v list number) → number' - 'crosMean(v list number) → number' - 'crosMin(v list number) → number' - 'crosMax(v list number) → number' - 'crosVar(v list number) → number' - 'crosStd(v list number) → number' - 'crosNorm(v list number) → list number'
```

English. AFRILANG library 'std/c/mlcrossval' (tier complex, category complex). Exports 7 function(s): crosSum, crosMean, crosMin, crosMax, crosVar, crosStd, crosNorm.

```
'import "std/c/mlcrossval"' · 'use mlcrossval'
```

```
- 'crosSum(v list number) → number' - 'crosMean(v list number) → number' - 'crosMin(v list number) → number' - 'crosMax(v list number) → number' - 'crosVar(v list number) → number' - 'crosStd(v list number) → number' - 'crosNorm(v list number) → list number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlcrossval' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : `croSum`, `croMean`, `croMin`, `croMax`, `croVar`, `croStd`, `croNorm`.

```
'import "std/c/mlcrossval"' · 'use mlcrossval'
```

```
- `croSum(v list number) → number`
- `croMean(v list number) → number`
- `croMin(v list number) → number`
- `croMax(v list number) → number`
- `croVar(v list number) → number`
- `croStd(v list number) → number`
- `croNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlcrossval"
use mlcrossval
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- `croSum(v list number) → number`
- `croMean(v list number) → number`
- `croMin(v list number) → number`
- `croMax(v list number) → number`
- `croVar(v list number) → number`
- `croStd(v list number) → number`
- `croNorm(v list number) → list`

4. Exemples d'utilisation

```
import "std/c/mlcrossval"
use mlcrossval
```

```
create result = croSum(/* args */)
say result
```

```
create result = croMean(/* args */)
say result
```

```
create result = croMin(/* args */)
say result
```

```
create result = croMax(/* args */)
say result
```

```
create result = crosVar(/* args */)
say result
```

```
create result = crosStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlcrossval"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlcrossval

```
function crosSum(v list number) returns number
end
```

```
function crosMean(v list number) returns number
end
```

```
function crosMin(v list number) returns number
end
```

```
function crosMax(v list number) returns number
end
```

```
function crosVar(v list number) returns number
end
```

```
function crosStd(v list number) returns number
end
```

```
function crosNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlcrossval' (tier complex, category complex). Exports 7 function(s): `crossSum`, `crossMean`, `crossMin`, `crossMax`, `crossVar`, `crossStd`, `crossNorm`.

'import "std/c/mlcrossval"' · 'use mlcrossval'

```
- `crossSum(v list number) → number`
- `crossMean(v list number) → number`
- `crossMin(v list number) → number`
- `crossMax(v list number) → number`
- `crossVar(v list number) → number`
- `crossStd(v list number) → number`
- `crossNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlcrossval"
use mlcrossval
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- `crossSum(v list number) → number`•
`crossMean(v list number) → number`•
`crossMin(v list number) → number`•
`crossMax(v list number) → number`•
`crossVar(v list number) → number`•
`crossStd(v list number) → number`•
`crossNorm(v list number) → list`

4. Usage examples

```
import "std/c/mlcrossval"
use mlcrossval
```

```
create result = crossSum(/* args */)
say result
```

```
create result = crossMean(/* args */)
say result
```

```
create result = crossMin(/* args */)
say result
```

```
create result = crossMax(/* args */)
say result
```

```
create result = crossVar(/* args */)
say result
```

```
create result = crosStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlcrossval"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

```
module mlcrossval
```

```
function crosSum(v list number) returns number
end
```

```
function crosMean(v list number) returns number
end
```

```
function crosMin(v list number) returns number
end
```

```
function crosMax(v list number) returns number
end
```

```
function crosVar(v list number) returns number
end
```

```
function crosStd(v list number) returns number
end
```

```
function crosNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlcrossval"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/mlcrossval/>

Source (extrait)

```
module mlcrossval

function crosSum(v list number) returns number
end

function crosMean(v list number) returns number
```

```
end

function crosMin(v list number) returns number
end

function crosMax(v list number) returns number
end

function crosVar(v list number) returns number
end

function crosStd(v list number) returns number
end

function crosNorm(v list number) returns list number
end
end
```